



Unidade 1 — Back-End e Front-End



Roteiro da Aula (50 min)

Tempo	Atividade
0-5 min	Abertura e contextualização
5-15 min	Revisão dos conceitos da Unidade 1
15-25 min	Discussão de um problema real
25-40 min	Demonstração prática
40-47 min	Desafio para os estudantes
47-50 min	Síntese e orientação da atividade



0-5 min: Contextualização

O que estudamos na Unidade 1:

- Aula 01: Introdução ao Desenvolvimento Web
- Aula 02: O que é Front-End?
- Aula 03: O que é Back-End?
- Aula 04: Como Front-End e Back-End se comunicam?
- Aula 05: AJAX e Comunicação Assíncrona
- Aula 06: Boas Práticas em Front-End e Back-End

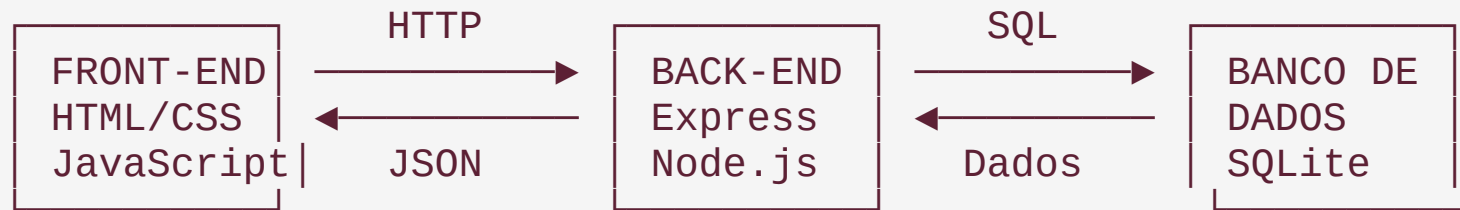
O que vamos fazer hoje:

Construir uma aplicação completa onde o Front-End consome uma API Back-End via `fetch()`, sem recarregar a página.



5-15 min: Revisão dos Conceitos

Fluxo de uma aplicação web:



Conceitos-chave:

- **Front-End:** HTML (estrutura) + CSS (estilo) + JS (comportamento);
- **Back-End:** servidor que processa requisições e acessa dados;
- **API:** contrato de comunicação via HTTP + JSON;
- **Fetch:** mecanismo assíncrono para consumir a API.



15–25 min: Problema Real

Cenário: Uma empresa precisa de um sistema para gerenciar contatos.

Requisitos:

1. Cadastrar contato (nome, telefone, e-mail);
2. Listar todos os contatos;
3. Editar um contato existente;
4. Excluir um contato.

Desafios técnicos:

- Como enviar dados do formulário para o servidor?
- Como atualizar a lista sem recarregar a página?
- Como tratar erros de validação?
- Como confirmar antes de excluir?



25-40 min: Demonstração Prática

Estrutura do projeto:

```
contatos-app/  
├── public/  
│   ├── index.html  
│   ├── style.css  
│   └── app.js  
├── server.js  
├── package.json  
└── .env
```

Passo 1: Configurar o servidor Express;

Passo 2: Criar as rotas da API;

Passo 3: Construir o Front-End;

Passo 4: Integrar com fetch().



Demonstração: Back-End (server.js)

```
const express = require('express');
const app = express();
app.use(express.json());
app.use(express.static('public'));

let contatos = [];
let idCounter = 1;

// Listar todos
app.get('/api/contatos', (req, res) => {
  res.json(contatos);
});

// Cadastrar
app.post('/api/contatos', (req, res) => {
  const { nome, telefone, email } = req.body;
  if (!nome || !telefone || !email) {
    return res.status(400).json({ erro: 'Todos os campos são obrigatórios' });
  }
  const contato = { id: idCounter++, nome, telefone, email };
  contatos.push(contato);
  res.status(201).json(contato);
});

// Excluir
app.delete('/api/contatos/:id', (req, res) => {
  const id = parseInt(req.params.id);
  contatos = contatos.filter(c => c.id !== id);
  res.status(204).send();
});

app.listen(3000, () => console.log('Servidor na porta 3000'));
```





Demonstração: Front-End (app.js)

```
const form = document.getElementById('form-contato');
const lista = document.getElementById('lista-contatos');

// Carregar contatos ao iniciar
async function carregarContatos() {
  const response = await fetch('/api/contatos');
  const contatos = await response.json();
  lista.innerHTML = contatos.map(c => `
    <li>
      <strong>${c.nome}</strong> - ${c.telefone} - ${c.email}
      <button onclick="excluirContato(${c.id})">Excluir</button>
    </li>
  `).join('');
}

// Cadastrar novo contato
form.addEventListener('submit', async (e) => {
  e.preventDefault();
  const dados = {
    nome: form.nome.value,
    telefone: form.telefone.value,
    email: form.email.value
  };
  await fetch('/api/contatos', {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify(dados)
  });
  form.reset();
  carregarContatos();
});

// Excluir contato
async function excluirContato(id) {
  if (confirm('Deseja realmente excluir?')) {
    await fetch(`/api/contatos/${id}`, { method: 'DELETE' });
    carregarContatos();
  }
}

carregarContatos();
```




40-47 min: Desafio

Desafio para os estudantes:

Adicione a funcionalidade de **edição** à aplicação.

Requisitos:

1. Ao clicar em "Editar", os dados do contato devem ser carregados no formulário;
2. O botão de envio deve mudar para "Atualizar";
3. Ao enviar, deve ser feito um `PUT` para `/api/contatos/:id` ;
4. Após atualizar, a lista deve ser recarregada.

Dica: Crie uma variável `editandoId` para controlar se está editando ou criando.



47-50 min: Síntese

O que fizemos hoje:

```
graph TD; A[Front-End (HTML + CSS + JS)] --> B[fetch() – Requisição HTTP assíncrona]; B --> C[Back-End (Node.js + Express)]; C --> D[Rotas REST (GET, POST, DELETE)]; D --> E[Dados em memória (array)]; E --> F[Resposta JSON]; F --> G[Front-End atualiza o DOM];
```

Atividade Prática Orientada: disponível na pasta da Unidade 1.

